

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра Систем сбора и обработки данных
(полное название кафедры)

Утверждаю

Зав. кафедрой ССОД

М.А. Бакаев
(подпись, инициалы, фамилия)

«___» _____ 202__ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Шмунк Александр Евгеньевич
(фамилия, имя, отчество студента – автора работы)

Разработка системы на основе нейросетевой модели для визуального анализа
(тема работы)

Состояния дорожного покрытия

Факультет автоматики и вычислительной техники
(полное название факультета)

Направление подготовки 09.03.02 – Информационные системы и технологии
(код и наименование направления подготовки бакалавра)

**Руководитель
от НГТУ**

Бакаев Максим Александрович
(фамилия, имя, отчество)

К.т.н., доцент
(ученая степень, ученое звание)

(подпись, дата)

**Автор выпускной
квалификационной работы**

Шмунк Александр Евгеньевич
(фамилия, имя, отчество)

АВТФ, АТ-14
(факультет, группа)

(подпись, дата)

Новосибирск 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра _____ *Систем сбора и обработки данных* _____
(полное название кафедры)

УТВЕРЖДАЮ

Зав. кафедрой _____ *Бакаев М.А.* _____
(фамилия, имя, отчество)

(подпись, дата)

**ЗАДАНИЕ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ БАКАЛАВРА**

Студенту _____ *Шмунк Александр Евгеньевич* _____
(фамилия, имя, отчество)

Направление подготовки *09.03.02 – Информационные системы и технологии* _____
(код и наименование направления подготовки бакалавра)

_____ *Факультет автоматики и вычислительной техники* _____
(полное название факультета)

Тема _____ *Разработка системы на основе нейросетевой модели для визуального анализа* _____
(полное название темы выпускной квалификационной работы бакалавра)
состояния дорожного покрытия

Исходные данные (или цель работы) _____ *Разработка системы для автоматического* _____
анализа состояния дорожного покрытия на основе нейросетевой модели

Структурные части работы _____

_____ *1. Анализ актуальности проблемы* _____

_____ *2. Изучение основ в области компьютерного зрения* _____

_____ *3. Изучение моделей нейронных систем* _____

_____ *4. Подготовка набора данных* _____

_____ *5. Обучение нейросетевых моделей* _____

_____ *6. Сравнение результатов* _____

7. Выводы

Задание согласовано и принято к исполнению.

**Руководитель
от НГТУ**

Бакаев Максим Александрович

(фамилия, имя, отчество)

К.т.н., доцент

(ученая степень, ученое звание)

(подпись, дата)

Студент

Шмунк Александр Евгеньевич

(фамилия, имя, отчество)

АВТФ, АТ-14

(факультет, группа)

(подпись, дата)

Тема утверждена приказом по НГТУ № _____ от «____» _____ 202__ г.
изменена приказом по НГТУ № _____ от «____» _____ 202__ г.

ВКР сдана в ГЭК № _____, тема сверена с данными приказа

(подпись секретаря государственной экзаменационной комиссии по защите ВКР, дата)

(фамилия, имя, отчество секретаря государственной
экзаменационной комиссии по защите ВКР)

Примечание: поля данного блока заполняются секретарем ГЭК при приеме ВКР к защите.

АННОТАЦИЯ

Данная выпускная квалификационная работа содержит 46 страниц, 21 рисунок, 2 таблицы и 22 источника.

Ключевые слова: дорожное покрытие, детекция, нейросеть, компьютерное зрение.

Цель работы является разработать систему на основе нейросетевой модели для визуального анализа состояния дорожного покрытия.

В первой главе проведен анализ актуальности проблемы дорожного покрытия, раскрыты понятия компьютерного зрения и методы обнаружения объектов на изображении.

Во второй главе были выбраны инструменты для разметки изображений и модель нейронной сети для обучения.

В третьей главе был подготовлен набор данных, на основе которых была обучена нейросеть. Проведен сравнительный анализ между архитектурами модели, а также продемонстрирована работа системы.

В результате выполнения ВКР была реализована система для анализа дорожного покрытия на основе нейросетевой модели.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1. Введение в проблему анализа дорожного покрытия. Понятие нейронных сетей и компьютерного зрения.	8
1.1. Актуальность и значимость мониторинга состояния дорог.....	8
1.2. Методы диагностики дорожного покрытия.....	9
1.3. Понятие нейронных сетей.....	11
1.3.1. Виды нейронных сетей.....	11
1.3.2. Методы обучения нейронных сетей.....	12
1.3.3. Свёрточная нейронная сеть.....	14
1.4. Основы компьютерного зрения.....	15
1.5. Методы обнаружения объектов на изображениях	16
1.6. Выводы по первой главе	18
2. Методы и инструменты реализации системы.....	20
2.1. Язык программирования	20
2.1.1. Python	20
2.1.2. Библиотеки	21
2.1.3. OpenCV и Ultralytics	21
2.2. Подходы для обучения нейронной сети.....	22
2.2.1. Выбор модели нейронной сети.....	23
2.2.2. Инструменты для разметки изображений	24
2.3. Метрики в задачах машинного обучения.....	25
2.4. Выводы по второй главе	28
3. Практическая часть.....	29
3.1. Датасет	29
3.1.1. Подготовка изображений	29
3.1.2. Разметка	31
3.2. Обучение нейронной сети.....	34

3.3 Проверка результатов.....	35
ЗАКЛЮЧЕНИЕ.....	41
СПИСОК ЛИТЕРАТУРЫ.....	42
ПРИЛОЖЕНИЕ	45

ВВЕДЕНИЕ

Современное состояние дорожной инфраструктуры играет ключевую роль в обеспечении безопасности дорожного движения, а также в эффективной логистике и транспортной доступности. Однако с течением времени дорожное покрытие подвергается износу, что приводит к появлению трещин, выбоин, неровностей и других дефектов.

Одной из главных задач в обеспечении безопасности дорожного движения является определение состояния дорожного покрытия и своевременная идентификация таких дефектов, например, как трещины и выбоины. Традиционные методы контроля состояния дорог, как правило, трудоёмкий и время затратный процесс.

С развитием технологий компьютерного зрения и нейросетевых подходов появилась возможность автоматизировать процесс мониторинга дорожного покрытия. Использование нейросетевых моделей для анализа изображений позволяет осуществлять точный и быстрый визуальный контроль состояния дорог на основе видеоданных.

Главной задачей работы является разработка нейросетевой модели для автоматического визуального анализа состояния дорожного покрытия. В процессе работы будет реализована система, способная выявлять и классифицировать дефекты на дорогах с использованием современных архитектур нейронных сетей.

Практическое значение данной разработки заключается в возможности её интеграции в системы интеллектуального мониторинга дорожной сети, что позволит значительно повысить эффективность планирования ремонтов и снизить риски, связанные с плохим качеством дорожного покрытия.

1. Введение в проблему анализа дорожного покрытия.

Понятие нейронных сетей и компьютерного зрения.

1.1. Актуальность и значимость мониторинга состояния дорог

Современные автомобильные дороги являются неотъемлемой частью в транспортной системе страны, обеспечивая связь между регионами, доступ к экономическим и социальным объектам, а также поддерживая внутреннюю и внешнюю торговлю. Качество дорожного покрытия напрямую влияет на уровень безопасности дорожного движения, скорость доставки грузов и комфорт передвижения. Однако в условиях климатических воздействий, высокой транспортной нагрузки и недостатка своевременного ремонта, дорожное полотно часто оказывается подвержено разрушениям, таким как трещины, выбоины и другим дефектам. Данные повреждения могут провоцировать потерю управления, особенно при высокой скорости движения, плохих погодных условиях или в ночное время суток. Это создает угрозу для участников дорожного движения и увеличивает риск дорожно-транспортных происшествий. Согласно официальным данным ГИБДД, в 2024 году в России произошло 126 705 ДТП, в результате которых пострадало более 170 тысяч человек [1]. Существенная часть этих происшествий, по оценкам экспертов, была связана с неудовлетворительным состоянием дорожного полотна.

Так же хочется затронуть состояние дорог в условиях сурового климата — особенно в северных и восточных регионах страны. В условиях низких температурах дорожное полотно замерзает, а при небольшом потеплении начинает частично оттаивать. За зимний сезон эти процессы повторяются многократно, создавая ощутимые напряжения внутри материала дороги. Со временем это приводит к образованию микротрещин и последующему разрушению покрытия. Длительное воздействие низких температур ослабляет связующие элементы в асфальте, делая его более уязвимым к механическим повреждениям. В итоге, под воздействием транспортных нагрузок, на поверхности дорог появляются неровности [2]. И если в течение

продолжительного времени никак не контролировать и не противодействовать данному процессу, то состояние дорожного покрытия будет ухудшаться.

Что касается отдаленных участках дорог, особенно в сельской местности или труднодоступных районах, то регулярный контроль состояния покрытия зачастую затруднён из-за нехватки специалистов, техники или финансовых ресурсов. Часто повреждения становятся заметны только тогда, когда уже достигли критических размеров. В результате повреждений становится все больше, увеличивая затраты на ремонт и приводя к необходимости капитального восстановления покрытия, вместо локального устранения дефектов на ранней стадии.

Из этого можно сделать вывод, что данная проблема является актуальной в наши дни. Контроль состояния дорожного покрытия и своевременное принятие мер по устранению опасных участков дорог необходим для обеспечения безопасного и комфортного вождения.

1.2 Методы диагностики дорожного покрытия

Способы оценки дорожного покрытия можно разделить на три категории:

1. Визуальный осмотр
2. Инструментальный метод
3. Лазерное сканирование

Визуальный осмотр является методом экспресс-диагностики и оценки состояния дорожного покрытия. Он позволяет выявить дефекты и отклонения на отдельных участках дороги. Визуальная оценка обычно проводится на начальной стадии обследования дорожного покрытия, а также при анализе качества ремонтных работ и содержания дорог. Визуальная оценка состояния дорожного покрытия осуществляется квалифицированными специалистами, обладающими необходимыми профессиональными навыками и опытом. Существует два основных метода визуального осмотра дороги. Первый метод предполагает

передвижение специалиста пешком или на транспортном средстве с низкой скоростью, при этом специалист делает остановки на различных участках дороги и проводит анализ состояния дорожного покрытия.

Во втором методе на автомобиль устанавливается видеокамера, которая фиксирует состояние дорожного покрытия во время движения. После съемки снятые кадры просматриваются на экране, отмечаются все дефекты, деформации и состояние дороги.

Метод, основанный на использовании специализированных инструментов, включает применение дорожной рейки. Данный прибор представляет собой трехметровую рейку с нанесенной измерительной шкалой и дополнительными устройствами для определения уклонов. Основная функция дорожного нивелира заключается в выявлении неровностей дорожного покрытия и основания автомобильных дорог, а также в измерении продольных и поперечных уклонов проезжей части. Преимущество данного метода заключается в его простоте и невысокой трудоемкости. Однако его реализация требует участия квалифицированного специалиста из дорожно-эксплуатационной службы.

Лазерный метод производится при помощи специального оборудования, которое устанавливается на транспортное средство. Система датчиков и камер, интегрированная в автомобиль, позволяет сканировать окружающую местность под углом 360 градусов в режиме реального времени. Результатом сканирования является формирование облака точек в трёхмерной системе координат. Точность и плотность данных зависят от конкретных требований к проведению съёмки. Процесс лазерной съёмки дорог состоит из двух основных этапов: непосредственно сканирования и последующей компьютерной обработки полученных данных. Для анализа и интерпретации результатов используются специализированные программные комплексы, которые позволяют проводить необходимые расчёты и

решать широкий спектр инженерно-технических задач. После обработки данные сопоставляются с аэрофотоснимками местности, что позволяет придать изображению высокую степень реалистичности. Ключевое преимущество лазерного метода — возможность проведения измерений без нарушения нормального функционирования транспортной инфраструктуры и без создания помех для движения транспортных средств [3].

Но в отличие от перечисленных методов, использование компьютерного зрения является относительно недорогим и менее трудозатратным способом диагностики, позволяющим автоматизировать процесс анализа дорожного покрытия. В следующих разделах более детально разберем виды нейронных сетей и компьютерное зрение.

1.3 Понятие нейронных сетей

Нейросеть — это компьютерная система, которая, подобно человеческому мозгу, способна учиться на основе информации, используя методы машинного обучения и состоя из множества связанных между собой «нейронов» [4].

1.3.1 Виды нейронных сетей

- Персептрон — это базовая и одна из старейших моделей искусственной нейронной сети. В своей простейшей форме (однослойный персептрон) он состоит из двух слоев: входного, куда поступает информация, и выходного, где информация обрабатывается и выдается результат. Обработка в выходном слое происходит с использованием функции активации
- Полносвязная нейронная сеть — это базовая архитектура нейронной сети, где каждый нейрон в одном слое связан со всеми нейронами в следующем. Она состоит из входного, выходного и, возможно, нескольких скрытых слоев. Эффективна для решения задач классификации и регрессии, и служит фундаментом для более сложных нейросетевых моделей.

- Рекуррентная нейронная сеть (RNN) специализируется на обработке последовательных данных, таких как текст или звук. Главная особенность - способность "запоминать" информацию о предыдущих элементах последовательности, что позволяет учитывать контекст.
- Сверточная нейронная сеть (CNN) предназначена для анализа изображений и видео. Ключевым элементом - сверточный слой, который с помощью фильтров извлекает важные признаки из входных данных.
- Генеративно-сопоставительная сеть (GAN) состоит из двух конкурирующих нейросетей: генератора, который создает новые данные, и дискриминатора, который пытается отличить сгенерированные данные от реальных. Широко используется для генерации изображений и улучшения качества существующих данных.
- Долгая краткосрочная память (LSTM) усовершенствованный тип RNN, способный удерживать информацию на протяжении длительных периодов времени. Применяется в задачах машинного перевода и анализа временных рядов.

Для реализации данной системы нужно будет использовать сверточные нейронные сети, так как предстоит работать с видео.

1.3.2 Методы обучения нейронных сетей

- Обучение с учителем

Это наиболее распространенный метод обучения. Нейронная сеть обучается на размеченных данных. Каждый пример содержит входные данные и соответствующий правильный ответ. Модель учится предсказывать правильный ответ для новых выходных данных. Данный метод применяется в задачах классификации, распознавания образов и регрессии. [5]

- Обучение без учителя

Тип машинного обучения, при котором модель работает с неразмеченными данными, то есть без заранее известных правильных ответов. Цель данной обучения выявить скрытые структуры, зависимости или группу в данных. [6]

- Метод Хэбба

Это закон обучения, основанный на работе Дональда Хэбба, который формулирует принцип взаимодействия между нейронами. Если два нейрона часто активируются вместе, связь между ними становится сильнее. В основе метода Хэбба лежит упрощенная модель нейронного взаимодействия, которая была актуальна на ранних этапах развития области нейронных сетей, но сейчас уже уступила место более сложным и продвинутым методам обучения. Важным следствием метода Хэбба является самоорганизация нейронов и возможность формирования представлений на основе опыта.

- Обучение методом соревнования

В этом методе обучения нейроны внутри нейронной сети соревнуются друг с другом за право представить входной сигнал. Во время обучения наиболее активные нейроны становятся "победителями" и приспособляются для лучшего представления входных данных. Этот метод обучения часто используется в самоорганизующихся картах Кохонена, которые позволяют обнаружить структуру и распределение входных данных без явного указания целевых значений.

- Генетические алгоритмы

Это глобальный метод оптимизации, вдохновленный естественным отбором и эволюцией. В этом методе обучения набор возможных решений (например, параметров нейронной сети) представлен популяцией, которая эволюционирует на протяжении нескольких поколений с использованием генетических операторов, таких как селекция, кроссовер (скрещивание) и мутация. Генетические

алгоритмы подходят для решения сложных задач оптимизации, где пространство решений большое и многоэкстремальное. Они могут использоваться как самостоятельный метод обучения нейронных сетей или в сочетании с другими методами для оптимизации гиперпараметров и структуры сети. Генетические алгоритмы способны находить хорошие решения в сложных задачах, хотя они могут быть менее эффективными по сравнению с градиентными методами, используемыми в обучении с учителем. [7]

1.3.3 Свёрточная нейронная сеть

Свёрточная нейронная сеть автоматически извлекают признаки из изображений, используя иерархическую структуру слоев. Это позволяет нейросетям научиться находить объекты.

Работа CNN проходит по такому сценарию:

- Процесс обработки изображений включает в себя применение последовательности сверточных слоев. Каждый из этих слоев выполняет операцию свёртки над входным изображением или выходом предыдущего слоя, используя набор фильтров, также известных как ядра свёртки. Эти фильтры выделяют определенные признаки, такие как вертикальные линии, углы и т. д.
- После сверточного слоя обычно применяется функция активации (например, ReLU).
- Между сверточными слоями часто используются pooling-слои, которые обеспечивают уменьшение размерности данных, сохраняя при этом наиболее важную информацию. Это повышает устойчивость сети к небольшим сдвигам и искажениям объектов, а также оптимизирует вычислительную сложность.
- В глубоких слоях нейронной сети осуществляется процесс комбинации простых признаков в более сложные, формируя иерархическое представление изображения. Таким образом, простые

признаки, такие как линии и углы объединяются в более сложные компоненты (части объектов).

- Выходные слои сети предсказывают классы объектов и параметры ограничивающих рамок (координаты, ширину, высоту).

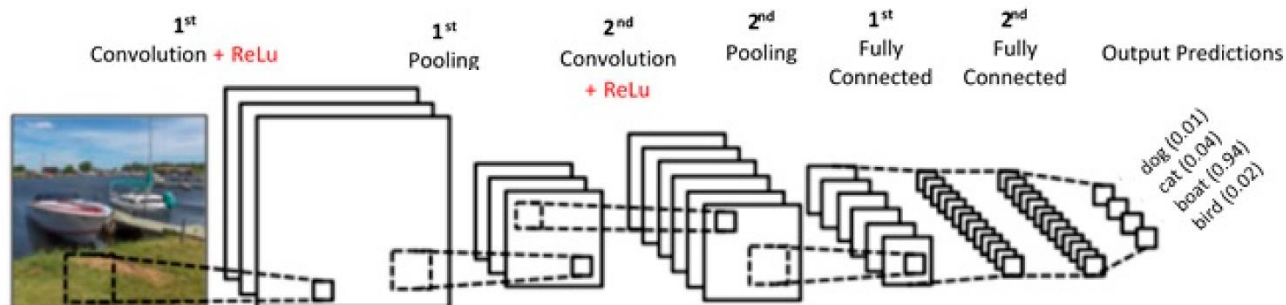


Рисунок 1 - Сверточная нейронная сеть [11]

1.4 Основы компьютерного зрения

Компьютерное зрение (сокращенно CV) — это технология, которая помогает компьютерам анализировать визуальную информацию: распознавать объекты, лица, текст и так далее.

Компьютерное зрение использует свёрточные нейронные сети, которые обучаются на больших наборах данных, чтобы научиться распознавать объекты.

Компьютер принимает визуальные данные в виде изображений или видео из камеры или другого устройства. Данные преобразуются в цифровую форму, которая представляет собой матрицу пикселей, что позволяет компьютеру осуществлять их обработку и анализ. С применением алгоритмов машинного обучения компьютер находит на изображении объекты, узоры или другие важные элементы. На основе проведенного анализа компьютер делает выводы: например, определяет, что на фото изображен человек, машина или другой интересующий объект [8].

Можно выделить три основные задачи компьютерного зрения. Ими являются классификация, детектирование и сегментация.

- Классификация изображений определяет, наличие

объекта интереса на изображении;

- Детектирование объектов является продолжением классификации, и выделяет область, в которой содержится объект интереса;
- Сегментация разделяет изображение на сегменты, сегменты одного типа, как правило, содержат объекты одного класса.



Рисунок 2 – Пример классификации, детекции и сегментации [9]

1.5 Методы обнаружения объектов на изображениях

Обнаружение объектов на изображении с использованием компьютерного зрения уже давно является неотъемлемой частью задач автоматизации и применяется во многих сферах жизнедеятельности, например:

- Медицина. Обнаружение объектов помогает автоматически выявлять новообразования, переломы и другие патологии на рентгеновском снимке, МРТ, КТ и так далее. Это помогает ускорить процесс диагностики и уменьшить вероятность ошибок.
- Промышленность. Детекция объектов используется для

распознавания дефектов, брака, контроля правильной сборки компонентов. Это повышает качество товаров, а также снижает затраты на ручной контроль.

- Беспилотный транспорт. Системы автономного вождения применяют технологию детекцию объектов для распознавания пешеходов, транспортных средств и других объектов. Это позволяет автомобилям выполнять автоматические торможения, перестроения, предсказывать действия других участников движения.

- Сельское хозяйство. Детекция на основе данных, полученных с дронов или видеокамер, используется для выявления состояния растений, присутствия вредителей, а также оценки зрелости и объема урожая [10].

На сегодняшний день существует большое множество моделей для детекции объектов, обычно их разделяют на две категории: двухстадийные и одностадийные модели.

- Двухстадийные модели.

Сперва они находят фрагменты изображения, где могут находиться объекты. Далее модель определяет класс объекта и его границы. Данные модели требуют много времени для вычислений, но результат получается точным.

Пример такой модели R-CNN. Получив на вход изображение, R-CNN начинает применять выборочный поиск для извлечения областей интереса (ROI), где каждая ROI представляет собой прямоугольник, который обозначает границу объекта на изображении. После этого каждая ROI проходит через нейронную сеть для получения выходных характеристик. Для каждого выходного признака ROI используется набор классификаторов методом опорных векторов, чтобы определить, какой тип объекта содержится в ROI [12].

- Одностадийные модели

В отличие от двухстадийных, данные модели сразу определяют его класс и положение на изображении минуя поиск промежуточных регионов. Благодаря такому подходу данные модели работают быстрее и подходят для задач реального времени.

Примером такой модели можно выделить YOLO. Основной идеей модели является объединение процесса разделения изображения и определение объектов в одном алгоритме. Для обнаружения объектов, YOLO использует сверточную нейронную сеть, которая позволяет находить объекты за один прогон картинки через нейросеть. Сначала картинка делится на сетку квадратов. Каждая клетка является “якорем”, к которому крепится bounding boxes. Получается, что вокруг клетки рисуется несколько прямоугольников, которые указывают на возможные объекты, и присваивает им вероятность наличия объекта и принадлежности к определённому классу. После получения предсказаний, запускается метод Non-Maximum Suppression, который фильтрует дублирующиеся и маловероятные рамки. В итоге на выходе модель получает только один прямоугольник обнаруженного объекта.

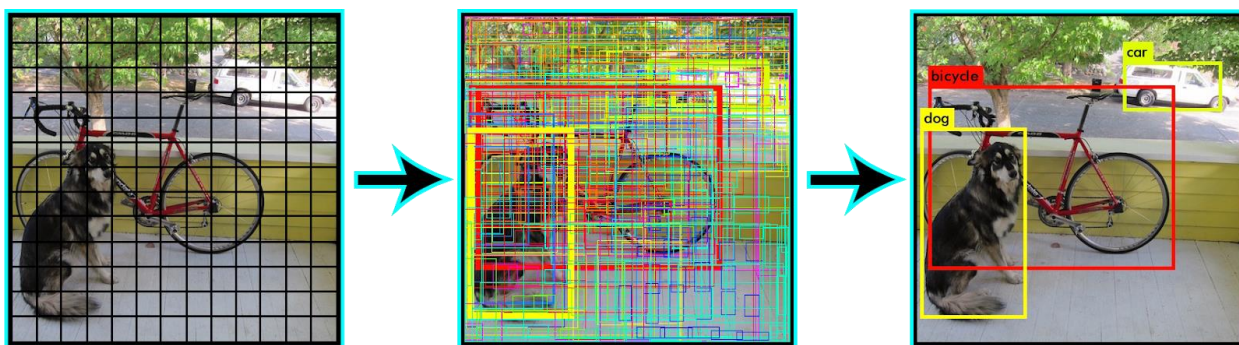


Рисунок 3 – Процесс обнаружения объектов с помощью Yolo [13]

1.6 Выводы по первой главе

Таким образом, в данной главе была раскрыта проблема дорожного покрытия. Был проведен обзор используемых на данный

момент классических методов определения состояния дорожного полотна. Также были рассмотрены виды нейронных сетей, в следствии чего была выделена нейросеть, которая будет использоваться в данной работе – сверточная нейронная сеть. Кроме того, было раскрыто понятие компьютерного зрения, рассмотрены наиболее актуальные задачи компьютерного зрения. Также был проведен обзор моделей обнаружения объектов на изображении. Для каждого из приведенных моделей было приведено краткое описание алгоритма работы.

2. Методы и инструменты реализации системы.

2.1 Язык программирования

2.1.1 Python

Для реализации данной работы был выбран язык Python, поскольку, на мой взгляд, он является оптимальным инструментом для работы с нейронными сетями. Можно выделить несколько факторов почему я выбрал этот язык:

- Python – интерпретируемый язык программирования, что позволяет выполнять программу без предварительной компиляции. Данная особенность значительно ускоряет процесс разработки и создания нейронных сетей.
- Python имеет активное и большое количество разработчиков, а также коллекцию разных библиотек, которые были разработаны для обучения и управления нейронными сетями.
- Синтаксис языка Python отличается своей легкостью в отличии от других языков программирования. Данная особенность помогает в упрощённом виде создавать и обучать нейронные сети. Такие библиотеки, как TensorFlow и Keras, предоставляют пользователю высокоуровневые интерфейсы, которые позволяют создавать сложные модели с минимальным объемом кода.
- В Python есть возможность создавать программы с различными парадигмами программирования, такие как: объектно-ориентированное, функциональное и аспектно-ориентированное программирование. Это позволяет выбирать наиболее подходящий подход для реализации конкретной нейронной сети.
- Поскольку Python является языком программирования с открытым исходным кодом, это позволяет создавать и использовать большое количество бесплатных библиотек, инструментов и фреймворков, которые могут быть полезны для разработки нейронной сети.

Python является отличным выбором для разработки нейронных сетей, потому что он обладает широкой функциональностью, простотой в использовании и постоянному развитию, что не мало важно, как для продвинутых, так и начинающих пользователей.

2.1.2 Библиотеки

Язык программирования Python включает в себя большое количество библиотек, которые помогают разработать нейронную сеть.

Приведу несколько примеров таких библиотек:

- **TensorFlow** — библиотека, разработанная компанией Google, с открытым исходным кодом, которая используется для машинного обучения, для решения задач построения и тренировки нейронной сети. С ее помощью разрабатываются системы для автоматического нахождения и классификации объектов.
- **PyTorch** — фреймворк, созданный компанией Facebook, предназначенный для машинного обучения. В него включены множество инструментов, которые используются для создания нейронных сетей, с помощью которых производится обработка компьютерного зрения, естественных языков и так далее. Вокруг фреймворка создана большое количество библиотек.
- **OpenCV** — это открытая библиотека, которая используется для работы с компьютерным зрением, обработкой изображений и видео.
- **Ultralytics** — это библиотека, с помощью которой пользователь может работать с моделями нейронной сети YOLO. Данная модель разработана для задач компьютерного зрения. Основана на PyTorch и имеет большой функционал, например: детекция, сегментация и классификация.

Данные библиотеки помогают пользователям в разработке нейронных систем и компьютерного зрения.

2.1.3 OpenCV и Ultralytics

В ходе разработки системы для визуального анализа состояния дорожного покрытия была выбрана комбинация библиотек OpenCV и

Ultralytics. Данное решение обусловлено их высокой эффективностью в области компьютерного зрения, а также интеграцию моделей глубокого обучения.

Ultralytics был выбран по следующим причинам:

- Библиотека обладает интуитивно понятным интерфейсом, что значительно упрощает процесс разработки нейронных сетей, особенно для начинающих.
- Ultralytics легко подстраивается под специфические задачи, позволяя, например, дообучать модели на пользовательских наборах данных.
- Ultralytics оперативно внедряет новейшие и наиболее эффективные архитектуры YOLO.
- Благодаря активному сообществу и подробной документации, решение проблем и поиск примеров использования значительно упрощаются.

Выбор OpenCV обусловлен следующими факторами:

- OpenCV обеспечивает широкие возможности для работы с изображениями и видео.
- Библиотека легко интегрируется с Python, Ultralytics, PyTorch и другими популярными инструментами машинного обучения.
- OpenCV является бесплатной библиотекой с открытым исходным кодом, активно поддерживаемой сообществом разработчиков, что гарантирует доступ к обширной документации и примерам.

Сочетание OpenCV и Ultralytics предоставляет мощную связку для эффективного решения задач анализа состояния дорожного покрытия на основе визуальных данных.

2.2 Подходы для обучения нейронной сети

Прежде чем непосредственно говорить про обучение нейронной сети, необходимо определить ключевые требования к разрабатываемой системе. В рамках поставленной задачи – визуальный анализ состояния

дорожного покрытия – важно брать во внимание несколько факторов. Система должна быть достаточно точной, чтобы надежно обнаруживать дефекты и при этом быстрой, так как, если обработка будет длиться какое-то продолжительное время, то от нее не будет смысла.

С учетом данных требований необходимо выбрать оптимальный подход к обучению, в том числе архитектуру модели и способ подготовки изображений. Одним из ключевых шагов на начальном этапе является создание размеченного набора данных, которое должно содержать изображения дороги с указанием местоположения и класса дефектов.

В следующих разделах речь пойдет про выбор архитектуры нейронной сети и инструмента разметки.

2.2.1 Выбор модели нейронной сети

На сегодняшний день существует большое количество моделей нейронных сетей, но больше всего из них выделяется YOLO, за счет своей высокой производительности и простоте использования. На рисунке 4 представлен график сравнения моделей, обученных на наборе данных COCO компании Microsoft. На графике mAP50-95 (mean average precision) это средняя точность модели при порогах уверенности от 0.5 до 0.95. Latency T4 TensorRT10 FP16 (ms/img) – время обработки одного изображения.

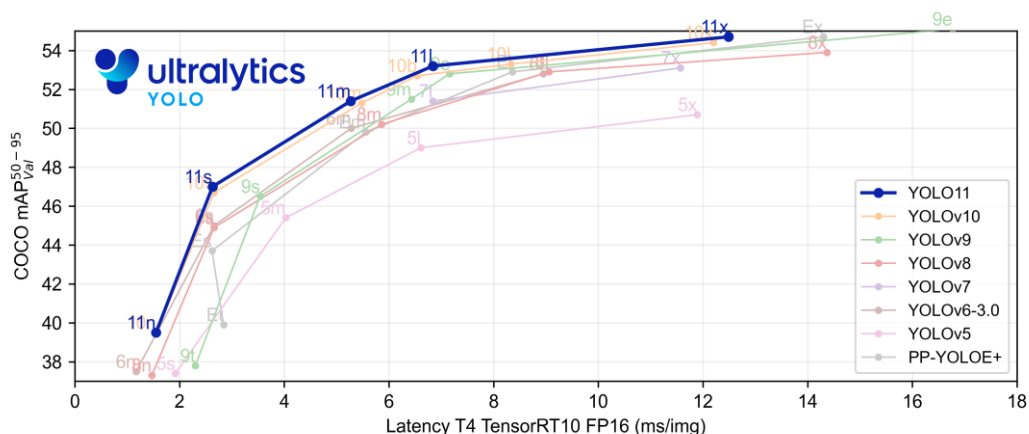


Рисунок 4 – График моделей YOLO [14]

Как видно из графика архитектура YOLO11 является точнее и быстрее чем прошлые аналоги. Плюс ко всему данная модель сохраняет более высокую среднюю точность на наборе данных COCO, используя на 22% меньше параметров чем прошлые модели [14].

Также существует 5 разновидностей моделей для каждой архитектуры. С возрастанием модели растет их точность, но падает скорость обработки изображения и обучения. На рисунке 5 представлены характеристики каждой модели. Можно заметить, что разница средней точности (mAP50-95) между YOLO11s и YOLO11m составляет всего 4.5, а скорость обработки на CPU больше в два раза.

Модель	размер (пикселей)	mAPval 50-95	Скорость CPU ONNX (мс)	Скорость T4 TensorRT10 (мс)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Рисунок 5 – Характеристики моделей YOLO11 [15]

Вследствие этих преимуществ было решено использовать YOLO11n и YOLO11s в данной выпускной квалификационной работе.

2.2.2 Инструменты для разметки изображений

Для корректной работы нейронной сети необходимо провести процесс первичного обучения. Этот этап включает в себя обучение сети по распознаванию изображений с заранее определёнными значениями. Это один из ключевых этапов в разработке эффективной системы анализа изображений. Важным компонентом данного процесса является разметка данных.

Разметка представляет из себя предварительную обработку изображений, с целью обеспечения её понятности для нейронной сети. В процессе изображения добавляются метаданные, содержащие свойства о характеристиках объектов.

Основная сложность данного процесса заключается в создании качественного набора исходных данных, или датасета. Для этого требуется разметить большое количество изображений по данному классу. Это позволяет увеличить вариативность данных и предотвратить возможные проблемы, связанные со «слепотой» нейронной сети в реальных условиях эксплуатации [16].

Точная и аккуратная разметка изображений является одним из главных аспектом в процессе обучения нейронной сети. Выделение определенного количества данных позволяет сфокусировать внимание на интересующий объект в кадре. Для задачи детекции разметку производят с помощью прямоугольников (bounding box).



Рисунок 6 – Пример размеченного изображения [17]

Для быстрой и комфортной разметки изображений существует множество инструментов, такие как: Labelimg, Labelbox, Roboflow, Cvat.

Для использования был выбран Cvat за счет своей универсальности и простоты.

2.3 Метрики в задачах машинного обучения

При решении задач машинного обучения, первым делом ставится вопрос оценки обученных моделей. Без конкретных критериев, нельзя будет оценить качество обучения модели, также как и сравнить разные модели между собой.

Именно поэтому важно учесть правильный выбор метрик для поставленной задачи, хотя множество существующих метрик может запутать и, в конечном счете, привести к неоптимальному решению [18].

Так же стоит затронуть значение матрицы ошибок.

Таблица 1 - Матрица ошибок

	$y = 1$	$y = 0$
$\hat{y} = 1$	True Positive (TP)	False Positive (FP)
$\hat{y} = 0$	False Negative (FN)	True Negative (TN)

Здесь $\hat{y}$ — это ответ алгоритма на объекте, а y — истинная метка класса на этом объекте. Таким образом, ошибки классификации бывают двух видов: False Negative (FN) и False Positive (FP) [19].

Оценка качества работы алгоритма может быть оценена метриками precision и recall.

Precision измеряет точность предсказаний алгоритма, процент правдивых предсказаний. Данная метрика вычисляется по формуле (1).

$$Precision = \frac{TP}{TP + FP} \quad (1)$$

Recall измеряет, насколько хорошо алгоритм может находить все положительные случаи. Данная метрика вычисляется по формуле (2).

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

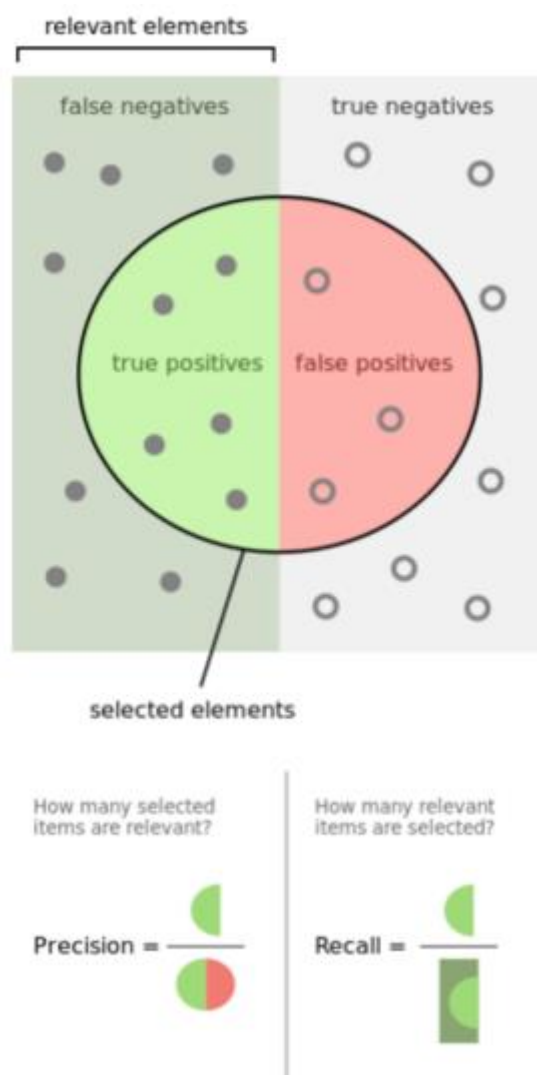


Рисунок 7 – Графическая интерпретация метрик precision и recall [19]

Отобразив зависимость precision от recall на графике, можно получить кривую как на рисунке 8. Найдя площадь под графиком, можно получить значение еще одной метрики: mAP (англ. mean average precision)— одна из популярных метрик, использующихся для оценки точности обнаружения объектов алгоритмами нейронных сетей.

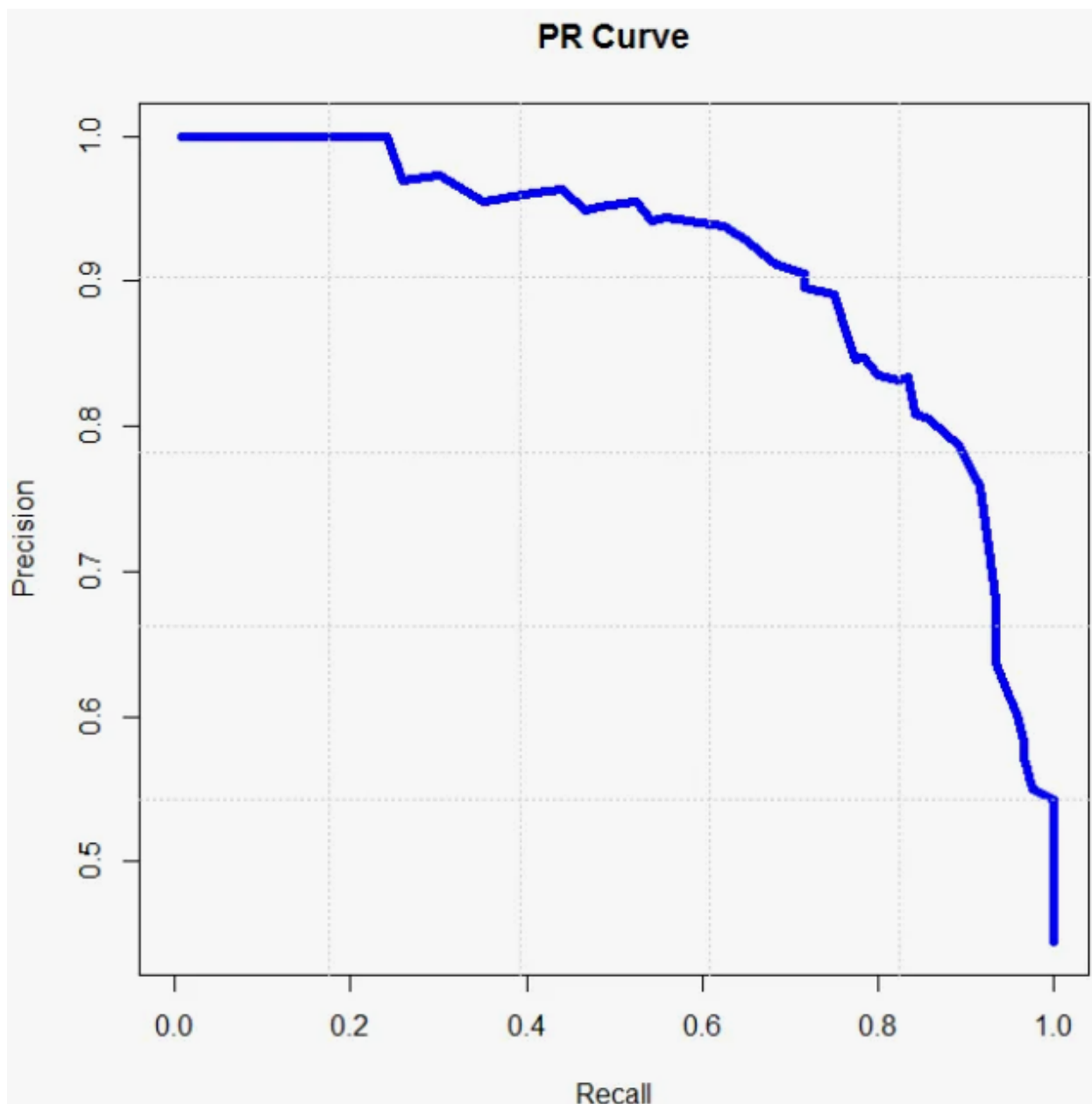


Рисунок 8 – Пример кривой precision-recall

2.4 Выводы по второй главе

В ходе данной главы был выбран язык программирования - Python. Был проведен обзор библиотек для Python, которые могут использоваться для реализации системы для анализа дорожного покрытия. Также был проведен выбор модели нейронной сети и инструментов, необходимых для создания датасета и последующего обучения нейросети на его основе. В добавок, было дано определение некоторым основным метрикам, на основании которых будет определяться качество обучения модели.

3 Практическая часть

Для проектирования данной системы был выбран язык программирования Python, а также дополнительные библиотеки к нему, такие как:

- Ultralytics – библиотека, предоставляющая удобный интерфейс для работы с моделями YOLO, предназначенными для задач компьютерного зрения.
- OpenCV – открытая библиотека, предоставляющая инструменты, связанные с компьютерным зрением, такие как анализ, классификация и обработка изображений.

3.1 Датасет

От того на сколько качественным окажется подготовленный датасет, будет зависеть качество обучения нейронной сети. Для достижения высокой точности обучение должно происходить на данных, максимально похожих на те, с которыми она будет сталкиваться в реальных условиях. Важно корректно выбирать изображения, например с разным уровнем освещения, погоду и так далее. Также важно правильно и ровно размечать объекты интереса.

3.1.1 Подготовка изображений

В рамках данной работы объектом интереса выступает дорожное покрытие, а конкретно дефекты на них. Для формирования набора данных производился поиск изображений через открытые источники и картографические службы, такие как Google и Яндекс карты. На рисунке 7 представлен пример изображения из датасета.

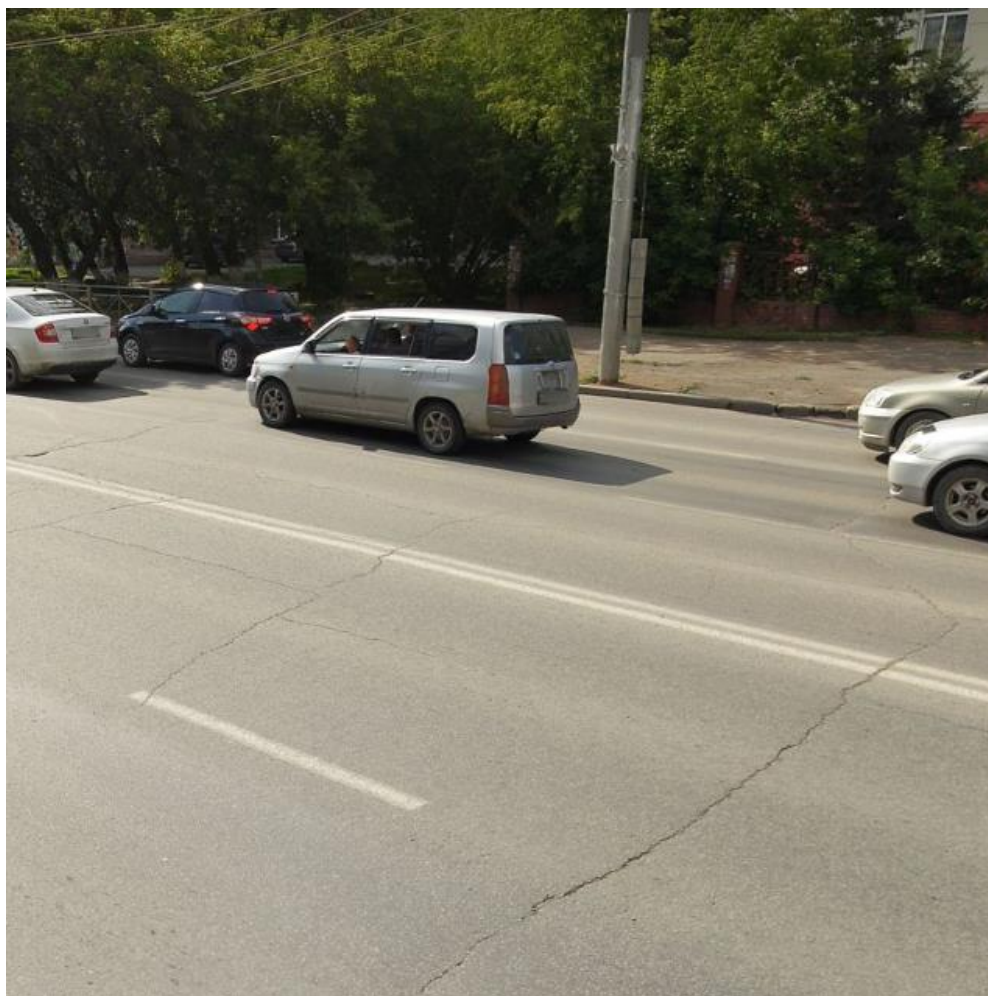


Рисунок 9 – Пример изображения дорожного покрытия с трещиной

Также были использованы изображения с хорошим состоянием дороги, чтобы нейросеть могла корректно отличать участки с дефектами от участков без повреждений. Это позволяет сформировать представление о том, как выглядит неповрежденная дорога и, следовательно, повысить точность.

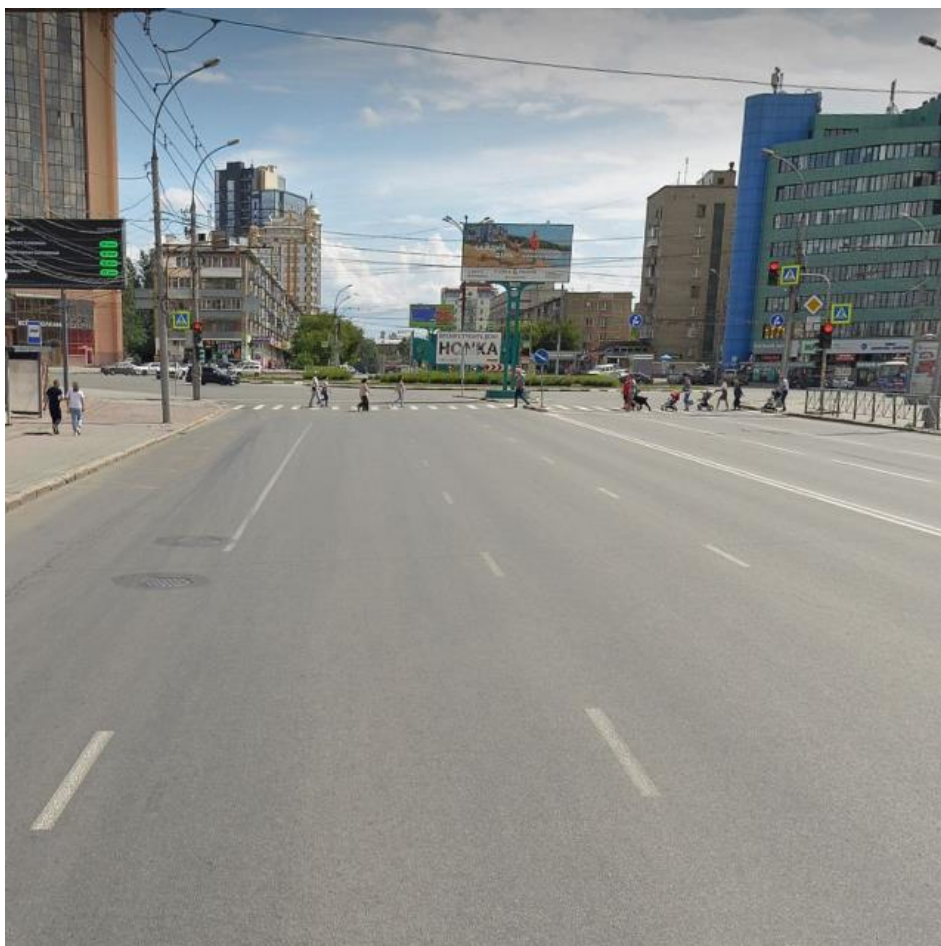


Рисунок 10 – Пример хорошей дороги

В конечном счете было собрано 900 изображений с разрешением около 640x640 так как это является значением по умолчанию в обучении моделей YOLO. Все изображения, которые не соответствуют данному разрешению, будут приведены к нему.

3.1.2 Разметка

Как говорилось ранее, в качестве инструмента для разметки был выбран Cvat. Перед началом работы 80% изображений было помещено в обучающий (train) и 20% в проверочный (validation) набор.

Обучающий набор (train) – это данные, которые используют, чтобы обучить модель, настроить параметры и найти закономерности.

Проверочный набор (validation) – данные, на которых нейросеть проверяет свое текущее состояние натренированности. Это происходит после каждой эпохи обучения, нейросеть пытается найти на проверочных

изображениях объекты, после чего сравнивает это с размеченными объектами [20].

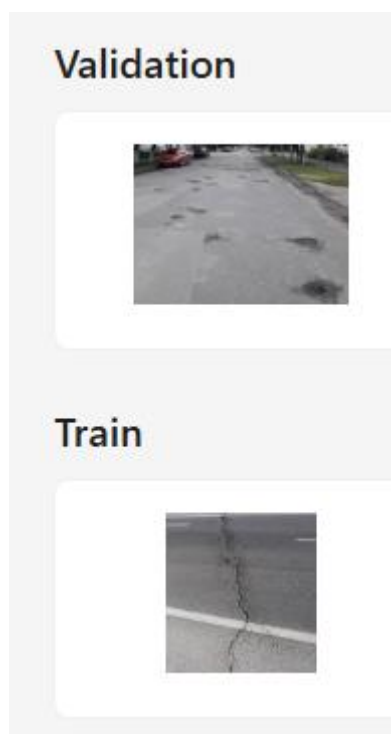


Рисунок 11 – Обучающий и проверочный наборы в Svat

В процессе разметки на каждом изображении вручную выделялись участки дефектов с указанием соответствующего класса. Было размечено 3 класса: трещины, выбоины и ямы, так как это самые часто встречаемые дефекты дорожного покрытия. Выделение повреждений производилось с помощью прямоугольников. На рисунке 10 показан пример разметки.



Рисунок 12 – Размеченная выбоина на изображении

После разметки всех фотографий был выгружен архив датасета с изображениями и текстовыми файлами координат ограничивающих рамок объектов.

Далее для работы понадобится файл `data.yaml`. Это файл, который содержит информацию о структуре датасета, необходимая для обучения модели YOLO. Он указывает на:

- Список названий классов
- Количество классов
- Путь к папкам `train` и `val`

Структура `data.yaml` файла показана на рисунке 11.

```
1  names:
2    - crack
3    - pothole
4    - hole
5  nc: 3
6  path: my_dataset_yolov
7  train: images/train
8  val: images/val
```

Рисунок 13 – data.yaml файл

3.2 Обучение нейронной сети

Обучение проводилось на нескольких моделях для выявления наиболее удачного варианта:

- 1) Модель без предварительного обучения;
- 2) Модель, предобученная на датасете RoadDamage [21], 700 изображений.

Для начала обучения нейронной сети используются следующие команды:

```
model = YOLO('yolo11s.pt')
results = model.train(data='my_dataset_yolov/data.yaml', epochs=100,
imgsz=640)
```

Эти команды запускают процесс обучения с заранее заданными параметрами. В первой строке указываем желаемую модель. Далее идет параметр data, который указывает путь к файлу data.yaml. Параметр epochs задает количество эпох – то есть полных проходов по обучающему датасету. Параметр imgsz – указывает размер изображения, которому они приводятся перед началом обучения.

Во время обучения данные обрабатываются не по одному изображению, а пакетами – так называемыми батчами (batch). Batch – это набор изображений, которые обрабатываются моделью одновременно [22]. Кроме того, в процессе обучения применяется мозаичная аугментация. Это один из видов расширения обучающей выборки, при котором несколько изображений объединяются в

одно. Это помогает лучше обобщать информацию и повышать ее устойчивость к различным условиям. Пример батча и мозаичной аугментации приведен на рисунке 12.

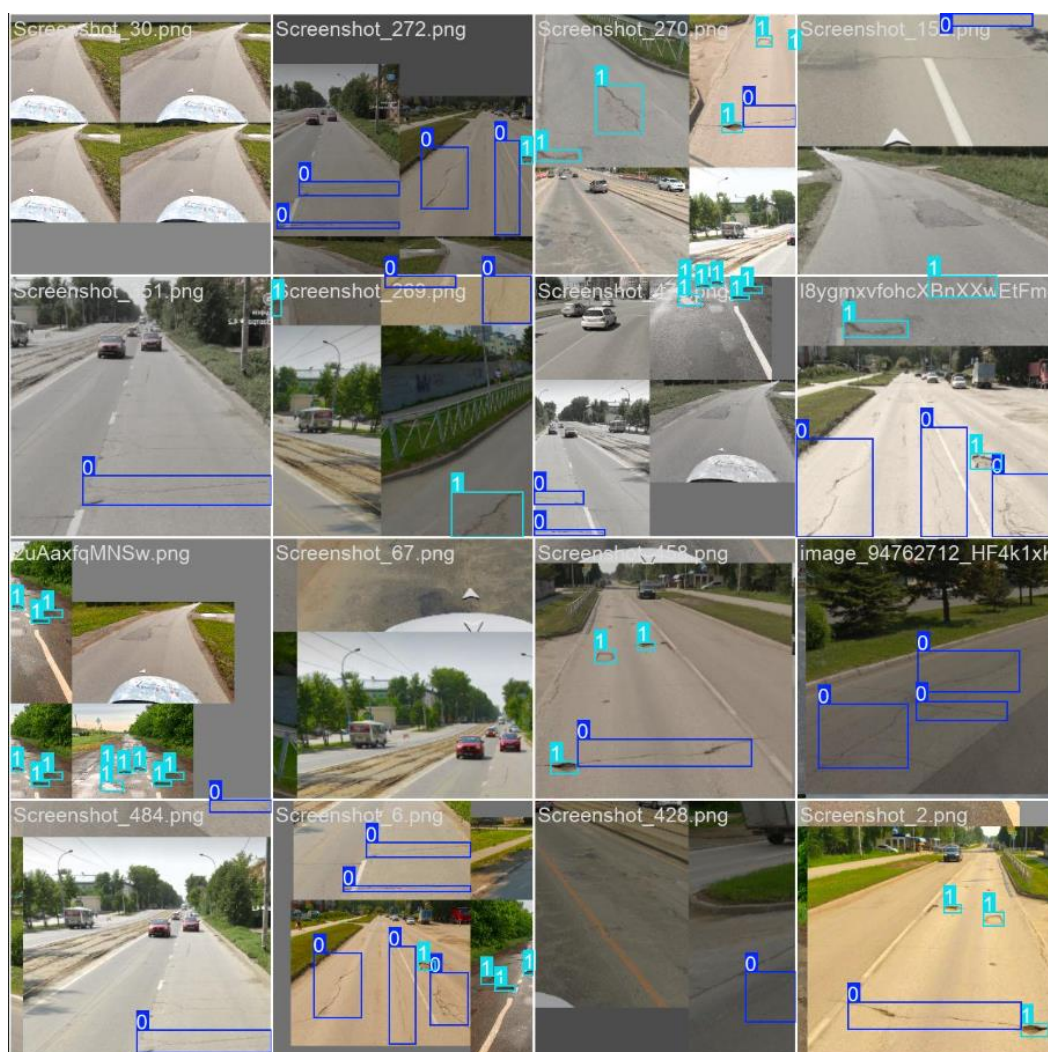


Рисунок 14 – Batch и мозаичная аугментация

3.3 Проверка результатов

После обучения в папке runs/detect/train в директории проекта появляются графики метрик, таких как F1-confidence, precision-confidence, precision-recall и recall-confidence. Все они помогают оценить, как работает модель, но самая информативная из них является precision-recall, поскольку она объединяет два параметра в один график и показывает, насколько точно (precision) модель справляется с нахождением объектов (recall). Далее будут приведены графики precision-recall для модели без предварительного обучения и предобученной на

датасете RoadDamage с использованием YOLO11n и YOLO11s со 100 эпохами обучения и YOLO11s с 200 эпохами обучения соответственно.

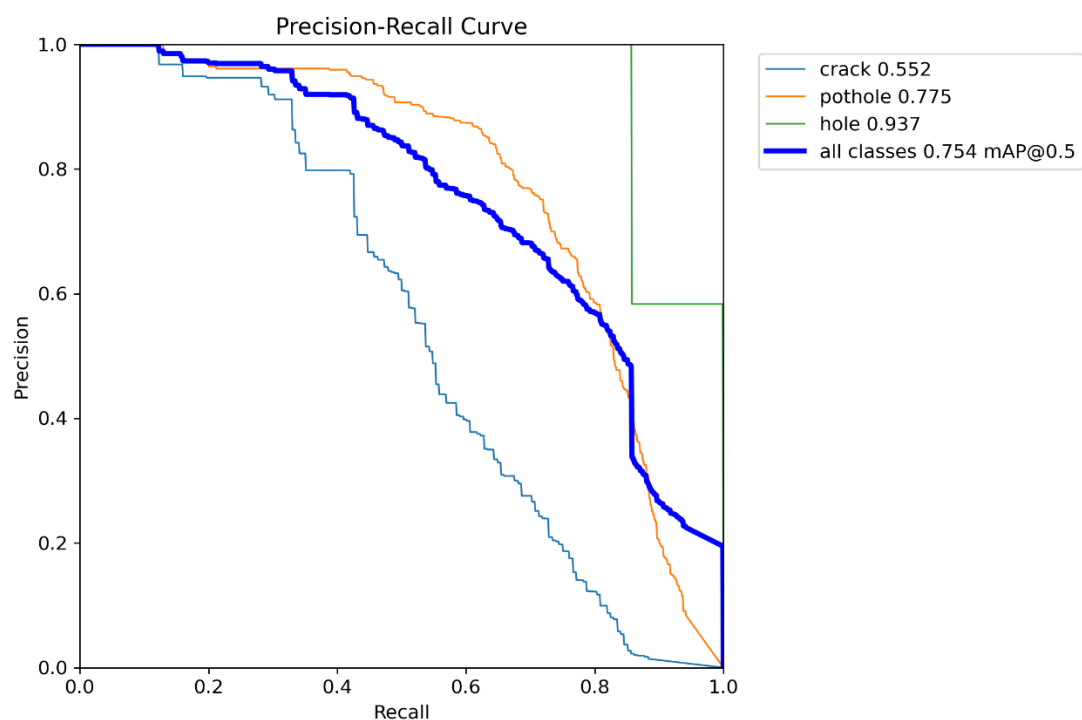


Рисунок 15 - График precision-recall для YOLO11n со 100 эпохами. Модель без предварительного обучения

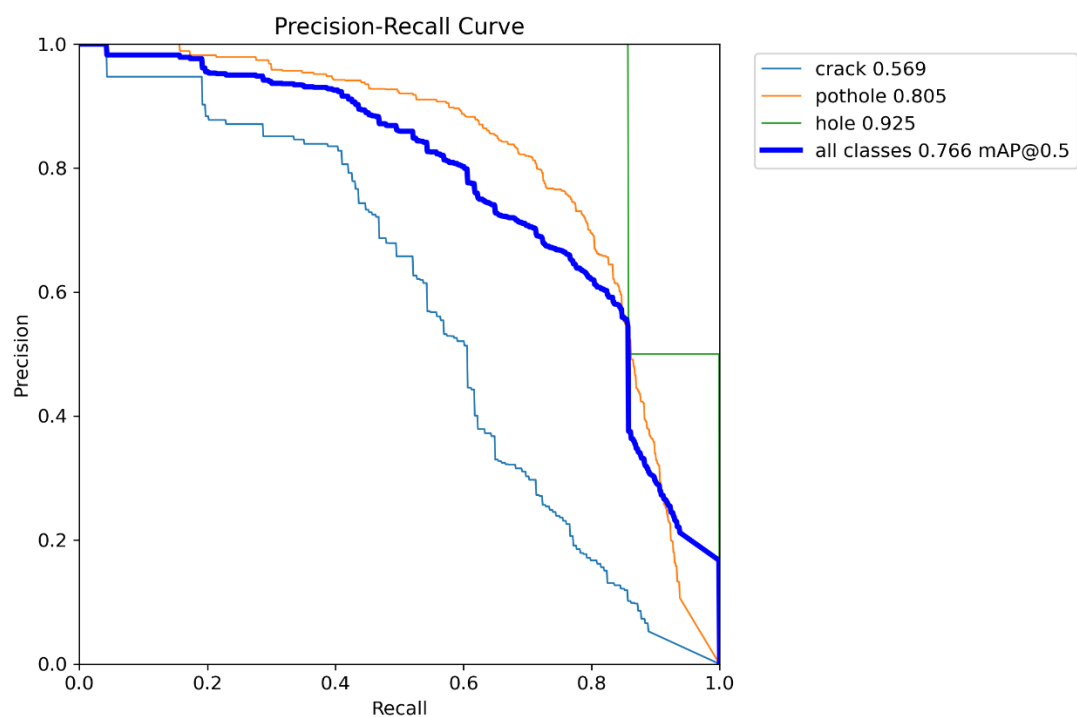


Рисунок 16 - График precision-recall для YOLO11s со 100 эпохами. Модель без предварительного обучения

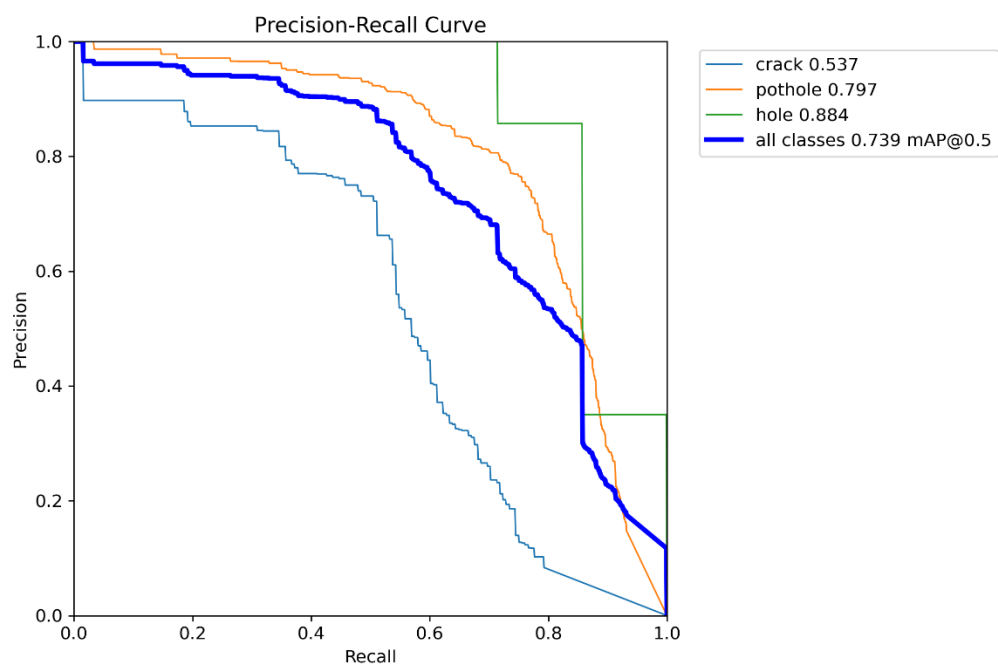


Рисунок 17 - График precision-recall для YOLO11s со 200 эпохами. Модель без предварительного обучения

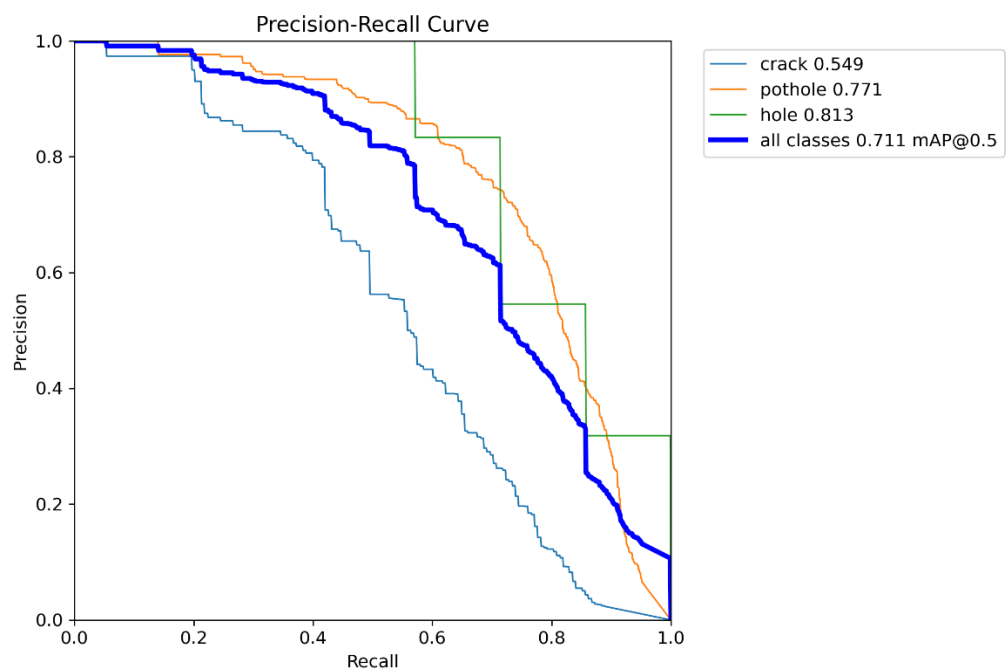


Рисунок 18 – График precision-recall для YOLO11n со 100 эпохами. Предобученная модель на датасете RoadDamage

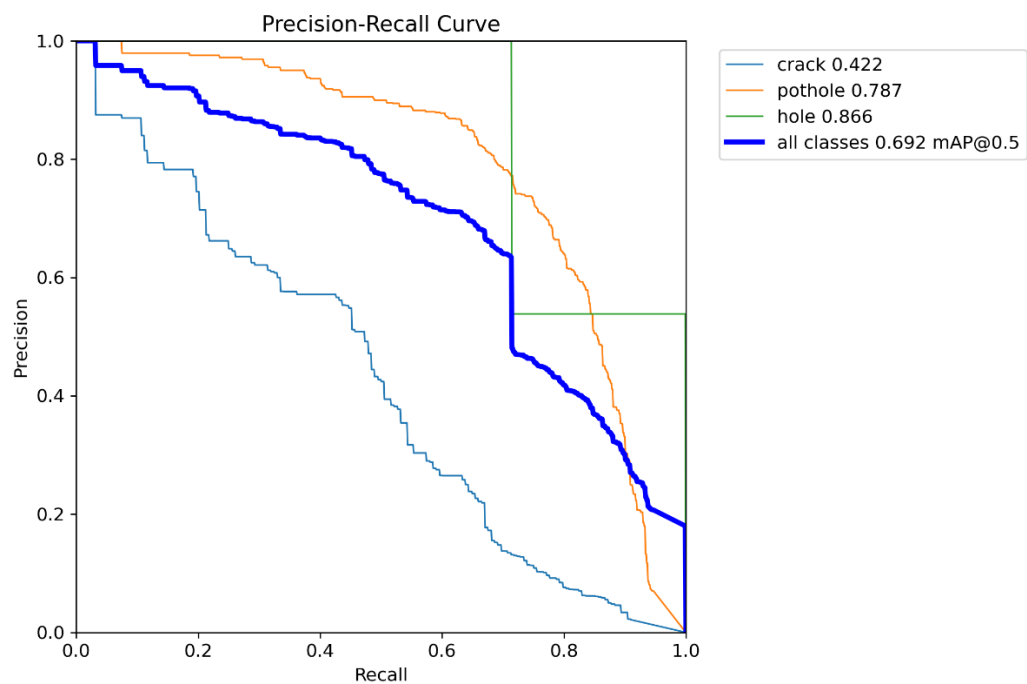


Рисунок 19 - График precision-recall для YOLO11s со 100 эпохами.

Предобученная модель на датасете RoadDamage

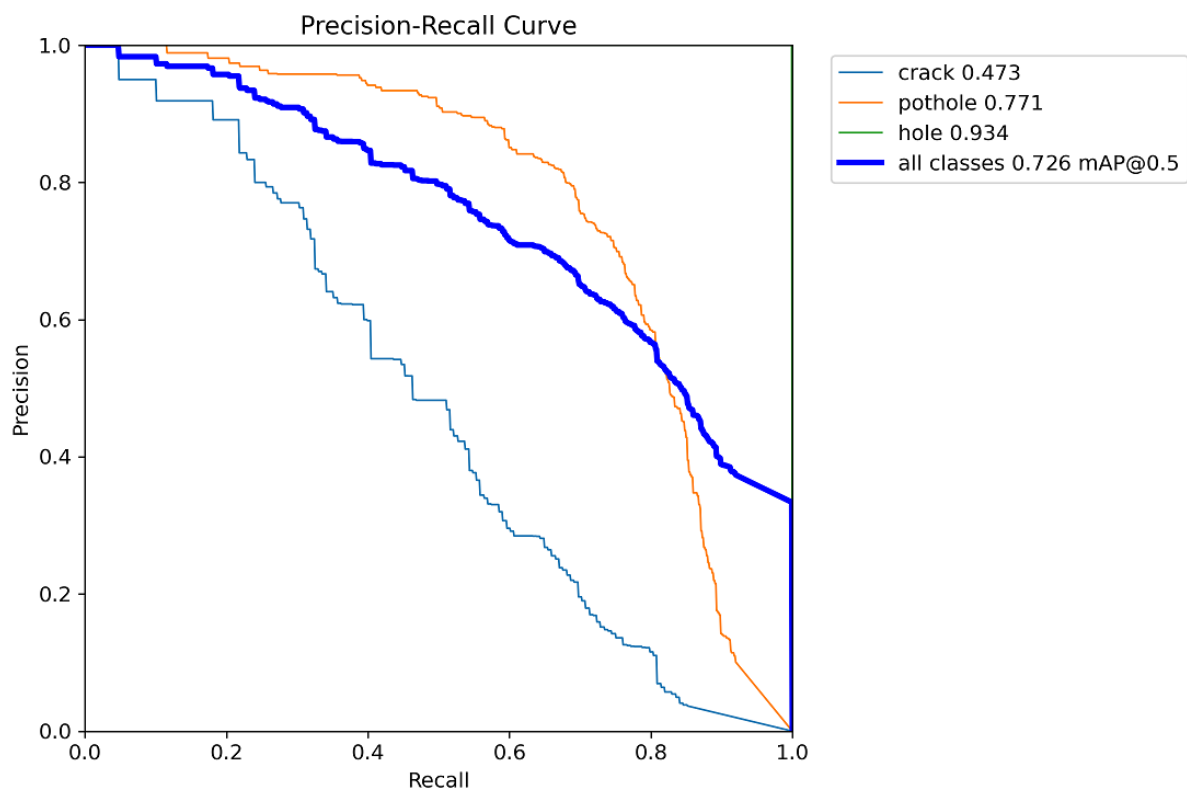


Рисунок 20 - График precision-recall для YOLO11s со 200 эпохами.

Предобученная модель на датасете RoadDamage

Качественной считается та модель, которая при увеличении полноты(recall), сохраняет высокие значения точности(precision).

Для наглядности, отобразим значения средней точности обученных моделей в таблице.

Таблица 2 - Средняя точность обученных моделей

	Модель без предварительного обучения	Предобученная модель
Nano, 100 эпох	0.754	0.711
Small, 100 эпох	0.766	0.692
Small, 200 эпох	0.739	0.726

Наилучший результат по метрике mAP@0.5 был получен у модели, обученной на собственном датасете на YOLO11s со 100 эпохами обучения и составил 0.766. Можно сделать вывод, что обучение модели с нуля на собственном датасете, адаптированном под конкретные условия и типы дорожных дефектов, обеспечивает более высокую точность по сравнению с использованием предобученных моделей.

На основе этой модели был реализован инференс (представленном в приложении) для демонстрации, как нейросеть локализует объекты и определяет их принадлежность к соответствующим классам. Для этого была разработана программа, которая запускает модель на видеопотоке и отображает найденные объекты в режиме реального времени. Это позволяет визуально оценить точность и стабильность работы модели. На рисунке 19 показан результат работы модели.

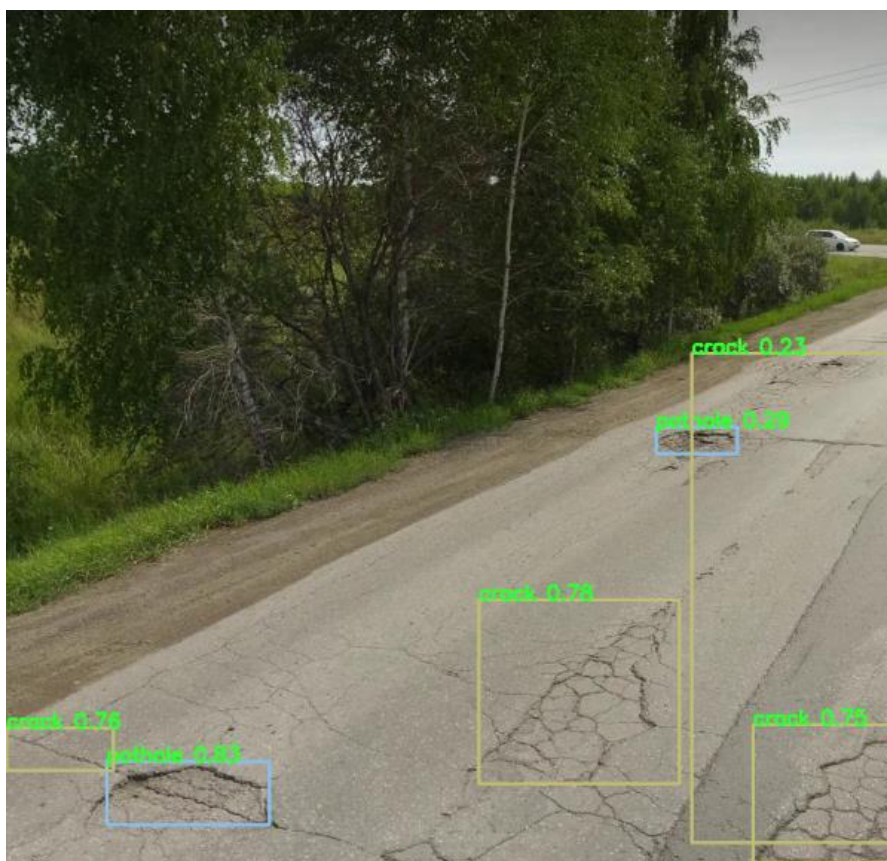


Рисунок 21 – Результат работы модели

ЗАКЛЮЧЕНИЕ

В данной работе рассматривалась разработка системы на основе нейросетевой модели, способная определять дефекты на дорожном покрытии в режиме реального времени.

В ходе работы был проведен анализ используемых на данный момент методов определения состояния дорожного покрытия, а также выбор необходимых инструментов для обучения моделей.

Обучение проводилось на моделях предварительно обученных на наборах данных и без предварительного обучения.

Удалось сформировать качественный датасет, максимально отражающий реальные условия. Это позволило добиться неплохих результатов точности по метрике mAP, особенно при обучении модели с нуля на собственных данных. Такой подход подтвердил важность соответствия обучающего материала задачам, решаемым в реальной среде. Дополнительно стоит отметить, что использование изображений, снятых в условиях, аналогичных предполагаемому применению модели, положительно сказалось на способности нейросети распознавать дефекты. В результате удалось создать устойчивую систему, демонстрирующую стабильные результаты в разнообразных ситуациях.

Хотя полностью заменить существующие методы оценки состояния дорожного полотна данная модель не сможет, однако она может помочь локализовать места с дефектами и сократить затраты времени на поиск. Применение модели в паре с видеорегистратором, установленным на транспортное средство или видеокамерами на улицах города может представлять собой крайне дешевый вариант обнаружения проблемных дорог.

СПИСОК ЛИТЕРАТУРЫ

1. Сведения о показателях состояния безопасности дорожного движения / [Электронный ресурс] // gibdd.ru: - URL: <http://stat.gibdd.ru/> (дата обращения: 25.04.2025)
2. Теплов А. Г. Полимерно-армированные композиты для дорожного строительства (ПАКДОР) [Электронный ресурс]. — 2024. — URL: <https://artem-teplov.ru/uploads/pdf/3.pdf> (дата обращения: 25.04.2025)
3. Сапрыкин, С. С. Методы диагностики состояния дорожного полотна / С. С. Сапрыкин, В. В. Пак, Д. А. Дегтярев. — Молодой ученый. — 2022. — № 15 (410). — С. 52-56. — URL: <https://moluch.ru/archive/410/90352/> (дата обращения: 27.04.2025).
4. The First Computational Theory of Mind and Brain: A Close Look at Mcculloch and Pitts's "Logical Calculus of Ideas Immanent in Nervous Activity". [Электронный ресурс]. — 2004. — URL: https://www.researchgate.net/publication/263265620_The_First_Computational_Theory_of_Mind_and_Brain_A_Close_Look_at_Mcculloch_and_Pitts's_Logical_Calculus_of_Ideas_Immanent_in_Nervous_Activity (Дата обращения: 27.04.2025)
5. Kröse B., Smagt P. An introduction to Neural Networks. University of Amsterdam [Электронный ресурс]. — 1996. — URL: <https://theswissbay.ch/pdf/Gentoomen%20Library/Artificial%20Intelligence/Neural%20networks/An%20Introduction%20to%20Neural%20Networks%20-%20Patrick%20van%20der%20Smagt.pdf> (дата обращения: 27.04.2025).
6. Smith L. An introduction to Neural Network. Unpublished draft, University of Stirling [Электронный ресурс]. — 2001. — URL: <https://studylib.net/doc/15072469/an-introduction-to-neural-networks> (Дата обращения: 28.04.2025)
7. Васенков Д. В. Методы обучения нейронных сетей [Электронный ресурс]. — 2007. — URL: <https://cyberleninka.ru/article/n/metody->

- obucheniyaiskusstvennyh-neyronnyh-setey/viewer (Дата обращения: 28.04.2025)
8. Компьютерное зрение: что это, где применяется [Электронный ресурс]. — 2025. — URL: https://www.reg.ru/blog/kompyuternoe-zrenie-cto-eto-gde-primenyaetsya/?utm_source=yandex.ru&utm_medium=organic&utm_campaign=yandex.ru&utm_referrer=yandex.ru (дата обращения: 27.04.2025).
 9. CV в медицине: как компьютерное зрение помогает лечить болезни [Электронный ресурс]. — 2025. — URL: <https://blog.skillfactory.ru/cv-v-medicine-kak-kompyuternoe-zrenie-pomogaet-lechit-bolezni/> (дата обращения: 30.04.2025).
 10. Детекция объектов в компьютерном зрении (CV) [Электронный ресурс]. — 2025. — URL: <https://data-light.ru/blog/detekciya-obektov-v-kompyuternom-zrenii/> (дата обращения: 30.04.2025).
 11. Debojit Biswas, Hongbo Su. An Automatic Car Counting System Using OverFeat Framework [Электронный ресурс]. — 2024. — URL: <https://www.mdpi.com/1424-8220/17/7/1535> (дата обращения: 02.05.2025).
 12. Region-Based Convolutional Networks for Accurate Object Detection and Segmentation [Электронный ресурс]. — 2024. — URL: <https://ieeexplore.ieee.org/document/7112511> (дата обращения: 05.05.2025).
 13. Real-Time-Human-detection [Электронный ресурс]. — 2024. — URL: <https://github.com/SSahas/Real-Time-Human-detection?tab=readme-ov-file> (дата обращения: 05.05.2025).
 14. Ultralytics YOLO11 [Электронный ресурс]. — 2024. — URL: <https://docs.ultralytics.com/models/yolo11/#supported-tasks-and-modes> (дата обращения: 06.05.2025).
 15. Ultralytics YOLO11 [Электронный ресурс]. — 2024. — URL: <https://docs.ultralytics.com/ru/models/yolo11/#key-features> (дата обращения: 07.05.2025).
 16. Сбор и разметка данных: как готовить, чтобы ваш ИИ говорил спасибо [Электронный ресурс]. — 2021. — URL: <https://vc.ru/future/302116-sbor-i->

- [razmetka-dannyh-kak-gotovit-chtoby-vash-ii-govoril-spasibo](#) (дата обращения: 10.05.2025).
17. CVAT: Speeding Up Image Annotation Since [Электронный ресурс]. — 2018. — URL: <https://www.edge-ai-vision.com/2019/09/cvat-speeding-up-image-annotation-since-2018/> (дата обращения: 10.05.2025).
18. Основные метрики задач классификации в машинном обучении [Электронный ресурс]. — 2020. — URL: <https://webiomed.ru/blog/osnovnye-metriki-zadach-klassifikatsii-v-mashinnom-obuchenii/> (дата обращения: 10.05.2025).
19. Метрики в задачах машинного обучения [Электронный ресурс]. — 2017. — URL: <https://habr.com/ru/companies/ods/articles/328372/> (дата обращения: 18.05.2025)
20. Тестовая, обучающая и валидационная выборки: зачем они нужны и как разделить данные [Электронный ресурс]. — 2024. — URL: <https://blog.skillfactory.ru/testovaya-obuchayuschaya-i-validatsionnaya-vyborki/> (дата обращения: 10.05.2025).
21. RoadDamage Computer Vision Project [Электронный ресурс]. — 2023. URL: <https://universe.roboflow.com/mtp-w2cxv/roaddamage-ytk79/dataset/2> (дата обращения: 10.04.2025).
22. Эпоха, батч, итерация — в чем различия? [Электронный ресурс]. — 2018. — URL: <https://neurohive.io/ru/osnovy-data-science/jepoha-razmer-batcha-iteracija/> (дата обращения: 10.05.2025).

ПРИЛОЖЕНИЕ

```
import cv2
from ultralytics import YOLO
import random

def draw_bounding_boxes_without_id(frame, results):
    boxes = results[0].boxes.xyxy.cpu().numpy().astype(int)
    classes = results[0].boxes.cls.cpu().numpy().astype(int)
    confidences = results[0].boxes.conf.cpu().numpy()
    for i, (box, cls) in enumerate(zip(boxes, classes)):
        if True:
            random.seed(int(cls)+8)
            color = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
            cv2.rectangle(frame, (box[0], box[1]), (box[2], box[3]), color, 2)
            conf = results[0].boxes.conf.cpu().numpy()
            cv2.putText(
                frame,
                f"{model.model.names[cls]} {conf[i]:.2f}",
                (box[0], box[1]),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.6,
                (50, 255, 50),
                2,
            )
    return frame

def process_video_with_tracking(input_video_path, show_video=True, save_video=False,
output_video_path="output_video.mp4"):
    cap = cv2.VideoCapture(input_video_path)
    if not cap.isOpened():
        raise Exception("Error: Could not open video file.")
    fps = int(cap.get(cv2.CAP_PROP_FPS))
    frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```

if save_video:
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_video_path, fourcc, fps, (frame_width, frame_height))
while True:
    ret, frame = cap.read()
    if not ret:
        break
    results_detect = model_detect.predict(frame, iou=0.4, conf=0.2, imgsz=608, verbose=False)
    if results_detect[0].boxes != None:
        draw_bounding_boxes_without_id(frame, results_detect)
    if save_video:
        out.write(frame)
    if show_video:
        frame = cv2.resize(frame, (0, 0), fx=0.75, fy=0.75)
        cv2.imshow("frame", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break
cap.release()
if save_video:
    out.release()
cv2.destroyAllWindows()
return results_detect

model = YOLO('runs/detect/train17/weights/best.pt')
model_detect = YOLO('runs/detect/train17/weights/best.pt')
model.fuse()
model_detect.fuse()

results_detect = process_video_with_tracking("2025-04-14 21-21-45.mp4", show_video=True,
save_video=False, output_video_path="output_video.mp4")

```